Ders 13 Çalışma Özeti

Ders 13 Çalışma Özeti

Genel Konular

- Seminer sunumları
  - Bu ders de öğrenci seminer sunumlarına ayrıldı. Her sunumda bir öğrenci veri sıkıştırmayla ilgili bir makaleyi anlattı; hoca sunum sırasında ve sonunda sorular sordu.
  - Yedi sunum yapıldı: sinir ağıyla kayıpsız sıkıştırma (NNCP), görüntü ve metinde RLE ile dinamik Huffman karşılaştırması, dinamik kümelemeli nicemleme ile kayıpsız ses sıkıştırma (DCQLAC), cep telefonu için BWT + Move-to-Front + aritmetik kodlama, araç verisi için maksimum entropi sinir ağı + optimize Huffman, XML sıkıştırma tekniklerinin karşılaştırması, akıllı sözlük tabanlı kodlama (IDBE).
- Sunum 1: NNCP, sinir ağlarıyla kayıpsız veri sıkıştırma
  - Amaç: bir sinir ağıyla kayıpsız sıkıştırma yapmak. Ağ, veriyi bloklar halinde işleyerek bir model öğrenir; sıkıştırma bu modelin tahminlerine dayanır. Makalede LSTM ve Transformer modelleri kullanılır.
  - enwik9 sonuçları (ekrandan, 00:10:00): gzip -9 322 591 995 bayt (2,58 bpb), xz -9 197 331 816 bayt (1,58 bpb), CMIX (v17) 116 394 271 bayt (0,93 bpb), Transformer 130 440 948 bayt (1,04 bpb), LSTM (large1) 128 292 351 bayt (1,03 bpb), LSTM (large2) 125 623 896 bayt (1,00 bpb).
  - Sunan öğrenci programı kendi bilgisayarında çalıştırdı (ekrandan, 00:40:00): XML içerikli 10 000 baytlık test dosyası 7 049 bayta indi (5,639 bit/bayt); Wikipedia'dan alınmış düz metin 5 541 bayttan 4 296 bayta indi (6,202 bit/bayt). Düz metin, tekrar eden etiketleri olan XML'e göre daha az sıkıştı.
- Sunum 2: Görüntü ve metin sıkıştırmada dinamik Huffman ve RLE
  - Kablosuz iletimde bant genişliği kısıtlı olduğundan veri sıkıştırılarak gönderilmek istenir. Görüntüler gri tonlamaya ve 64x64 boyutuna indirilir; metinlerde karakterlerin ASCII karşılıkları kullanılır.
  - Görüntüde RLE hem daha yüksek sıkıştırma oranı hem daha kısa süre verdi (ekrandan, 00:26:40). Metinde kısa metinlerde dinamik Huffman, uzun metinlerde RLE daha iyi sıkıştırdı; RLE her durumda daha hızlıydı (ekrandan, 00:30:00).
  - Aynı yazarların ses çalışmasında (ekrandan, 00:41:15) iki yöntemin sıkıştırma oranı neredeyse aynı (yaklaşık %87,2-87,5), ama RLE çok daha yavaş (65-116 sn; Huffman 2-2,7 sn).
- Sunum 3: Dinamik kümelemeli nicemleme ile kayıpsız ses sıkıştırma (DCQLAC)
  - Kayıpsız ses sıkıştırma savunma, biyomedikal ve adli alanlarda gereklidir; müzik iletiminde kayıplı yöntemler yaygındır.
  - Literatür (ekrandan, 00:50:00): MPEG-4 ALS (2003), MPEG-4 ALS RLS-LMS (2004), MPEG-4 SLS (2006), CELP (2010), Sparse Linear Predictor (2013), OLS-NLMS (2014), FLAC-MD5 (2017), WavPack (2020), Monkey's Audio (2020).
  - Yöntem dört aşamalıdır: örnekleme (Nyquist ölçütü, $f_s > 2f_{max}$), dinamik kümeleme (genliğin dinamik aralığı küme sayısına bölünür), her kümede nicemleme, sonra BWT ve Huffman ile bit akışının sıkıştırılması. Kod çözme ters sırada yapılır.
  - Sonuçlar (ekrandan, 01:01:45): beş şarkı kategorisinde genel ortalama sıkıştırma oranı Monkey's Audio %57,83, WavPack %53,89, FLAC %71,30, DCQLAC %95,31.
- Sunum 4: Cep telefonu için metin sıkıştırma (BWT, Move-to-Front, aritmetik kodlama)
  - Cep telefonundaki metin uygulamaları (bir hadis uygulaması) için dört düzen karşılaştırıldı: yalnız aritmetik kodlama; MTF + aritmetik; BWT + aritmetik; BWT + MTF + aritmetik.

  - Sıkıştırma oranı $\frac{\text{sıkıştırılmış boyut}}{\text{orijinal boyut}} \times 100$ olarak tanımlandı; küçük olan daha iyidir.
  - En iyi sonuç üçünün birlikte kullanılmasıyla, 8192 baytlık BWT bloğunda elde edildi (yaklaşık %36,2). Büyük blok, MTF'de sıfıra dönüşecek daha çok karakter demektir.
- Sunum 5: Araç terminali verisi için maksimum entropi sinir ağı ve optimize Huffman
  - Toplu taşıma araçlarındaki terminaller konum, hız, plaka gibi verileri kablosuz olarak sunucuya gönderir. Terminalin donanımı zayıftır; büyük model çalışmaz, buluta bağlanmak gizlilik riski taşır.
  - Maksimum entropi ilkesine uyan istatistiksel model iki katmanlı bir sinir ağı olarak kuruldu; bu model Huffman ağacının kurulmasını optimize etmek için kullanıldı.
  - Sonuçlar (ekrandan, 01:44:10): sıkıştırma oranı Huffman ~%49, LZW ~%55, önerilen yöntem ~%59; aktarım süresi, iletim hata oranı ve paket kaybı da en düşük önerilen yöntemde.
- Sunum 6: XML sıkıştırma tekniklerinin karşılaştırılması
  - Sınıflandırma: genel metin sıkıştırıcılar (gzip, bzip2, PPM) ve XML'in farkında olan sıkıştırıcılar; bunlar da şema gerektiren ve gerektirmeyen, sorgulanabilen (homomorfik ve homomorfik olmayan) ve sorgulanamayan olarak ayrılır.
  - XMill etiketleri ve veriyi ayrı sıkıştırır; XGrind (Huffman) ve XPress (aritmetik kodlama) homomorfik, XQzip homomorfik değildir.
  - Sonuç: XML dosyalarında en iyi genel performansı gzip verdi; ardından XMill'in gzip arka uçlu sürümü ve PPM geldi. gzip ise sıkıştırılmış veri üzerinde sorguya izin vermez.
- Sunum 7: Akıllı sözlük tabanlı kodlama (IDBE)
  - Yıldız kodlaması (*-encoding), bir sözlük yardımıyla kelimelerdeki harflerin çoğunu * ile değiştirir; böylece BWT'den önce metin daha tekrarlı hale gelir (ekrandan, 02:08:35).
  - IDBE, metinden bir sözlük kurar ve kelimeleri kısa kodlarla değiştirir; sonra BWT tabanlı sıkıştırma uygulanır.
  - Calgary derleminde örneğin bib dosyası (108,7 KB) için BWT 2,11, BWT + *Encode 1,93, BWT + IDBE 1,69 bit/karakter (ekrandan, 02:13:25).

Hocanın Özellikle Vurguladığı Kısımlar

- Kayıpsız sıkıştırmada hata ölçütü
  - Kayıpsız sıkıştırmada MSE sıfır olmalıdır. Sıfırdan farklı çıkıyorsa bu yöntemden değil iletim kanalının gürültüsünden gelir; o zaman görüntü ile metin arasında fark olmamalıdır, çünkü kanal için ikisi de bir dosyadır.
- RLE'nin metin uzunluğuna bağlı davranışı
  - RLE ardışık tekrarları kodlar; metnin kısa ya da uzun olması karakter düzeyinde bunu değiştirmemelidir. Hoca, karakterlerin ASCII bitlerine çevrilip 1 ve 0'ların ardışıklığına RLE uygulanmış olabileceğini önerdi. Aynı yazarların ses çalışmasındaki RLE adımları gerçekten bit düzeyinde ardışık 1 ve 0 sayımıdır (ekrandan, 00:41:15).
- Sıkıştırma oranı aynıysa
  - İki yöntem aynı oranda sıkıştırıyorsa süre öne çıkar; seste dinamik Huffman bu yüzden tercih edilir.
- Kaynak seçimi
  - Araştırmada ve tezde dergi makalelerine öncelik verilmelidir. Bildirilerin hakem süreci kongreden kongreye çok farklıdır; dergi daha sıkı bir değerlendirmeden geçer.
  - Bir çalışmanın "gelecek çalışmalar" kısmında yazılanların sonradan yapılıp yapılmadığına bakmak önemlidir.
- Sıkıştırma oranının tanımı
  - Makalelerde sıkıştırma oranı farklı tanımlanır (kalan boyut oranı ya da kazanç oranı). Sonuçları yorumlarken hangi tanımın kullanıldığına bakılmalıdır.

Kısa Tekrar Notları

- NNCP: sinir ağı tabanlı kayıpsız sıkıştırıcı; enwik9'da LSTM (large2) 1,00 bpb ile gzip'in (2,58 bpb) çok altında, CMIX'e (0,93 bpb) yakın.
- Tekrarlı yapıdaki veri (XML) düz metne göre daha iyi sıkışır: NNCP denemesinde 5,64 ve 6,20 bit/bayt.

- Görüntüde RLE dinamik Huffman'dan iyi ve hızlı; metinde kısa metinlerde Huffman, uzun metinlerde RLE iyi; seste oranlar eşit, Huffman çok daha hızlı.
- Renkli görüntüyü gri tonlamaya çevirmek, renk bilgisini kaybettirdiği için kendi başına kayıplı bir işlemdir.
- DCQLAC: örnekleme, dinamik kümeleme, kümede nicemleme (adım $\Delta = 2U_m/L$, $L = 2^n$, eşik $\Delta^2/12$), BWT ve Huffman.
- SNR ve PSNR'de yüksek değer daha iyidir.
- BWT kendisi sıkıştırmaz; aynı karakterleri yan yana getirip sonraki aşamayı (MTF, RLE, entropi kodlayıcı) kolaylaştırır ve tersine çevrilebilir.
- MTF sık tekrarlanan sembolleri küçük indekslere (çoğunlukla 0) çevirir.
- BWT + MTF + aritmetik kodlama birlikte en iyi sonucu verir; BWT blok boyutu büyüdükçe sıkıştırma iyileşir.
- bzip2 sırası: RLE, BWT, MTF, RLE, Huffman. gzip: LZ77 + Huffman (deflate).
- Homomorfik XML sıkıştırıcı orijinal yapıyı korur ve sıkıştırılmış veride sorguya izin verir.
- Yıldız kodlaması ve IDBE, BWT öncesi metni sözlükle dönüştürerek bit/karakter değerini düşürür; bedeli ek süredir.

Detaylı Açıklamalar

- NNCP ve sinir ağıyla sıkıştırma
  - Sinir ağı tabanlı sıkıştırmada model, verideki bağlamdan sonraki sembolün olasılığını öğrenir; sıkıştırma bu olasılıklara göre yapılır. Büyük veriyle eğitilen model daha başarılıdır ve sıralı veri ile zaman serisi analizinde de kullanılabilir.
  - Hoca giriş katmanına neyin verildiğini (örneğin 512 baytlık bir blokta her karakterin bir nörona gidip gitmediğini) sordu; sunumda net bir cevap verilmedi. Hoca LSTM'in bir önceki kelimelerden sonraki kelimenin gelme olasılığını öğrendiğini, sıkıştırmanın bu olasılıkları kullanıyor olabileceğini belirtti.
  - Öğrencinin çalıştırdığı sürümün parametrelerinde model Transformer'dır ve `n_layer=4` görünür (ekrandan, 00:40:00). Sunumda "STEP" sütunundaki 19 ve 10 değerleri katman sayısı olarak yorumlandı.
  - **Ders sonrası not:** Ekrandaki çıktıda katman sayısı `n_layer=4` olarak ayrıca verildiği için "STEP" sütunu katman sayısı değildir; büyük olasılıkla işlenen eğitim/kodlama adımlarının sayısıdır.
  - Bit/bayt değeri sıkıştırılmış boyutun 8 ile çarpılıp orijinal boyuta bölünmesidir: $7049 \cdot 8/10000 \approx 5{,}64$ ve $4296 \cdot 8/5541 \approx 6{,}20$.
- RLE ve dinamik Huffman karşılaştırması
  - Görüntü sonuçları (ekrandan, 00:26:40): Leena.png için Huffman CR %7,727 (0,596 sn), RLE CR %47,406 (0,206 sn); Parrot.jpg için RLE %63,672. Tabloda MMSE ve PSNR sütunları her görüntü için tek değerdir; hoca, iki yöntemin oranları farklıyken PSNR'ın ikisi için aynı olamayacağını, değerin hangi yönteme ait olduğunun yazılması gerektiğini belirtti.
  - Tablodaki CR'nin nasıl tanımlandığı makalede verilmemiş. Makale RLE'nin "daha iyi sıkıştırma oranı" verdiğini söylediğine göre bu tabloda yüksek değer daha iyi anlamına gelir.
  - Metin sonuçları (ekrandan, 00:30:00): Example1-3 (kısa) için Huffman CR %59,1, %58,1, %81,25; RLE %44,3, %47,8, %47,9. Example4-6 (uzun) için Huffman %45,0, %42,7, %42,5; RLE %43,2, %44,4, %46,3.
  - Çalışma 2012 dolaylarında bir uluslararası konferans bildirisidir. Hoca içeriği zayıf bulduğunu, bu soruların bir dergi makalesinde yanıtlanmış olması gerektiğini söyledi.
- DCQLAC ayrıntıları
  - Nicemlemede ilk kümede nicemleme biti 2 seçilir, MSQE hesaplanıp eşikle karşılaştırılır; eşiğin altında değilse işlem tekrarlanır, sonra seviyeler ikili bitlerle kodlanır (ekrandan, 00:56:25). Kalan kümelere aynı adımlar uygulanır.
  - BWT aşamasında nicemlenmiş sinyalin bit akışı 5 bitlik gruplara ayrılır, 32 gruba BWT uygulanır ve sonuca Huffman uygulanır; stereo seste iki kanal için aynı işlem yapılır (ekrandan, 00:58:20).
  - Slayttaki BWT örneği: `ABCDDCBADBCA$` girdisinin bütün döndürmeleri sıralanır ve son sütun alınır; sonuç `AC$BCDABDBADC` olur (ekrandan, 00:58:20; python ile doğrulandı).

  - Kod çözmede Huffman ağacında kökten başlanır, bit 0 ise sağa, 1 ise sola gidilir (makaledeki kural), yaprakta karakter yazılır; sonra ters BWT, nicemleme bitinin bulunması ve kümelerin genlik aralıklarına göre birleştirilmesi gelir (ekrandan, 01:00:00).
  - Bir öğrenci 2 bitlik nicemlemenin kayba yol açıp açmayacağını sordu; sunan öğrenci kümelerin parçalarının parçaları kodlandığı için 2 bitin yettiğini söyledi.
  - **Ders sonrası not:** Sunumda SNR'ın yüksek olmasının kötü olduğu söylendi; doğrusu SNR'da da PSNR'da olduğu gibi yüksek değer daha az bozulma, yani daha iyi sonuç demektir.
- Cep telefonu için BWT + MTF + aritmetik kodlama
  - Aritmetik kodlama örneği (ekrandan, 01:11:40): "FATIMAH AS" metninde 10 karakter vardır; A 3/10, diğerleri 1/10 olasılıklıdır. Aralıklar: boşluk [0,00; 0,10), A [0,10; 0,40), F [0,40; 0,50), H [0,50; 0,60), I [0,60; 0,70), M [0,70; 0,80), S [0,80; 0,90), T [0,90; 1,00).
  - MTF örneği (ekrandan, 01:12:50): "aaafhhimst" dizisi alfabe listesiyle kodlanınca 0, 0, 0, 5, 7, 0, 8, 12, 18, 19 elde edilir (python ile doğrulandı).
  - Makaledeki BWT tablosunda "fathimahas" döndürmeleri son karaktere göre sıralanır ve çıktı "aaafhhimst" olur (ekrandan, 01:14:15).
  - **Ders sonrası not:** Standart BWT'de döndürmeler baştan sıralanır ve son sütun alınır; "fathimahas" için çıktı `mhfsathiaa` olur (python ile). Makaledeki tabloda çıktı yalnızca harflerin sıralanmış hali olduğundan, ek bilgi olmadan tersine çevrilemez; tablo büyük olasılıkla hatalı ya da basitleştirilmiştir.
  - Sonuçlar: yalnız aritmetik kodlamada en iyi oran %58, en kötü %75; MTF + aritmetikte %36 ve %54; BWT + aritmetikte %59 ve %76 (blok boyutları 1024, 2048, 4096, 8192). Üçü birlikte en iyi sonucu verdi.
- Araç verisi ve optimize Huffman
  - Sinir ağıyla sıkıştırmanın sorunu verinin iki kez taranmasıdır: önce model eğitilir, sonra veri kodlanır. Önerilen yöntemde maksimum entropi ilkesine uyan bir olasılık modeli iki katmanlı sinir ağıyla gerçekleştirilir; çıkış y_i sonraki karakterin i olma olasılığıdır.
  - Optimize Huffman yedi adımdan oluşur; ilk adımda keyfi bir kök düğüm seçilip ağırlığı 0 yapılır (ekrandan, 01:40:00). Sembol akışının taranma sayısı bire iner ve bellek ihtiyacı azalır. Hoca adımların makalede örneksiz verildiğini ve tam anlaşılamadığını belirtti.
  - Deneyde 300 dakikalık veri 5, 10 ve 15 saniyelik çevrimlerle gönderildi. Tablodaki sıkıştırma oranı $1 - \frac{\text{sıkıştırılmış}}{\text{orijinal}}$ biçimindedir: 15 sn için $1 - 336{,}43/824{,}56 \approx \%59{,}2$ (ekrandan, 01:44:10; python ile doğrulandı).
  - Bir otobüsten 4 günde toplanan 12 600 hız verisinin 239'u kayboldu, 12 361'i iletildi; ortalama kare hata 0,23 çıktı ve hız verisinin %98,1'i doğru iletildi.
- XML sıkıştırma
  - XML her kayıtta şemayı tekrarladığı için kendini açıklayıcıdır, ama bu tekrar dosyayı büyütür ve sıkıştırmaya çok elverişli hale getirir (ekrandan, 01:50:00).
  - XGrind örneği (ekrandan, 02:00:00): her etikete T0, T1, … ve her niteliğe A0 gibi kısa kod verilir; veri Huffman ile sıkıştırılır, Huffman ile sıkıştırılmayan alanlara `nahuff` başlığı eklenir. Yapı korunduğu için homomorfiktir.
  - XQzip yapıyı değiştirip ortak düğümleri birleştirir ve ek indeks ekler; bu yüzden homomorfik değildir.
  - Veri setinde "data ratio", etiketler atıldığında kalan verinin oranıdır; rastgele üretilmiş dosyalarda 0, Wikipedia dosyalarında 0,91-0,98'dir (ekrandan, 02:02:00).
  - **Ders sonrası not:** Sunumda PPM "Partial String Matching" diye açıldı; doğru açılımı "Prediction by Partial Matching"tir.
- IDBE ve yıldız kodlaması
  - Yıldız kodlamasında sözlükteki her kelime, olabildiğince çok harfi * olan bir koda dönüştürülür (örneğin "But" `B**`, "soft" `*of*`). Daha çok yıldız daha iyi sıkıştırma demektir. Sözlüğün nasıl kurulduğu sunumda anlatılmadı.
  - IDBE'de metindeki kelimelerden sözlük kurulur ve kelimeler kısa kodlarla değiştirilir.
  - Calgary sonuçlarında IDBE çoğu dosyada en düşük bit/karakter değerini verdi, ama geo dosyasında BWT'den kötüdür (5,18 ve 4,84) (ekrandan, 02:13:25). Süre yalnız BWT'den uzun, yıldız kodlamasından kısadır.
  - **Ders sonrası not:** Sunumda değerler "bit per second" diye okundu; tablodaki

birim BPC, yani karakter başına bittir.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.